Software Architecture and Design Specification

Project: Online Quiz System

Version: 1.0

Authors: Debangana, Chinmay, Satvik, Sunil

Date: 28-09-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	28-09-2026	Debangana, Chinmay, Satvik, Sunil	Initial architecture and design specification for the Online Quiz System

Approvals

Role	Name	Signature/Date
Course Coordinator		
Project Guide		
Team Lead		

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design of the Online Quiz System. It describes the major components, their responsibilities, interactions, interfaces, error handling, security design, and traceability to the SRS.

1.2 Scope

The design covers the console-based multiple-choice quiz application, including quiz start, question display, answer submission and validation, score calculation, result display, quiz restart, and administrator question management. The system is designed for a single user at a time and does not include network or multi-user functionality.

1.3 Audience

Students, developers, testers, instructors, and project evaluators.

1.4 Definitions

MCQ – Multiple Choice Question; UI – User Interface; SRS – Software Requirements Specification; SAD – Software Architecture and Design Specification; RTM – Requirements Traceability Matrix.

2. Document Overview

2.1 How to use this document

Section 3 describes the system architecture, component diagram, architecture pattern, technology choices, risks, requirement traceability, and security architecture. Section 4 describes the detailed design, UML sequence diagrams, internal APIs, error handling, logging, and user experience. Section 5 provides the glossary and references.

2.2 Related Documents

Software Requirements Specification (SRS) – Online Quiz System; Software Engineering Mini-Project Deliverables Part-1.

3. Architecture

3.1 Goals & Constraints

Goals:

• Provide a simple and clear console-based quiz flow.

• Calculate and display scores correctly.

• Keep quiz, input, and question-data responsibilities separate.

• Allow administrators to maintain the question bank.

• Handle invalid input and missing/invalid question data without crashing.

Constraints:

• Console-based application only.

• C++17 and standard library are used.

• Local question data is used; no network or multi-user operation is required.

• The project is developed within the academic mini-project scope.

3.2 Stakeholders & Concerns

Stakeholder	Concerns	Addressed by
Player/User	Clear instructions, easy answer entry, correct score	Console UI, Quiz Controller, Quiz Logic
Administrator	Question creation and maintenance	Admin Module and Question Data
Developer/Tester	Modularity and testability	Layered architecture and separate components
Instructor/Evaluator	Requirement traceability and clear design	Traceability section, UML diagrams and documented interfaces

3.3 Component (UML) Diagram

The component diagram shows the main modules of the Online Quiz System and the direction of their dependencies.

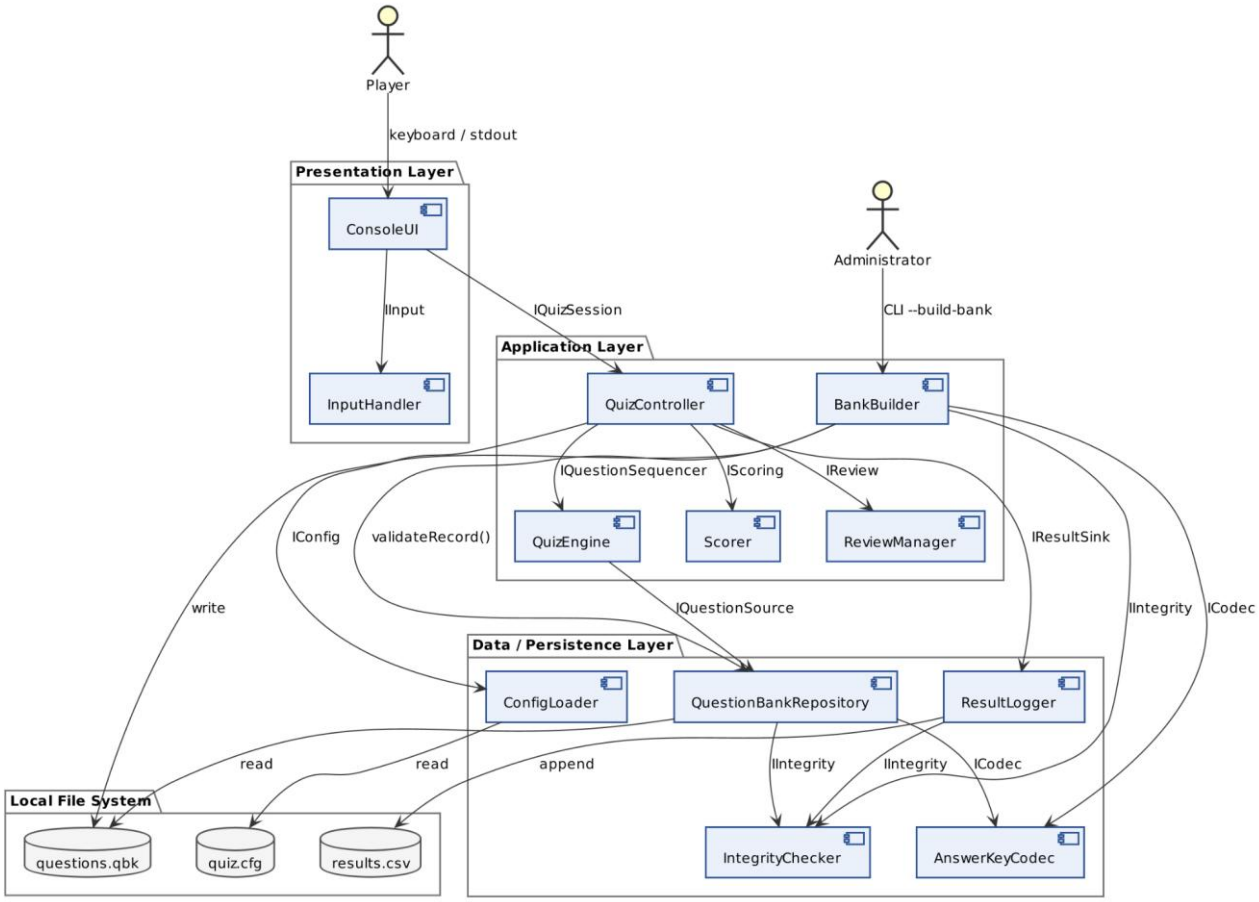


Figure 3.1 – UML Component Diagram

3.4 Component Descriptions

Component	Layer	Responsibility	Related Requirements
Console UI	Presentation	Displays menus, questions, options, instructions and final results.	OQ-F-001, OQ-F-003, OQ-F-008, OQ-F-016, OQ-NF-001
Quiz Controller	Application	Controls quiz flow, question progression, restart and coordination between UI and quiz logic.	OQ-F-001, OQ-F-011, OQ-F-014, OQ-F-016
Quiz Logic	Application	Retrieves questions, checks answers and calculates quiz results.	OQ-F-003, OQ-F-006, OQ-F-007, OQ-F-008, OQ-F-009
Question Data	Data	Stores and provides the questions and correct answers used by the quiz.	OQ-F-002, OQ-F-013, OQ-SR-002, OQ-SR-003
Score / Result	Application/Data	Maintains score information and prepares the final result for display.	OQ-F-007, OQ-F-008, OQ-F-010
Admin Module	Application	Provides administrator functions for adding, modifying and deleting quiz questions.	OQ-F-012, OQ-F-013, OQ-F-015, OQ-SR-005

3.5 Chosen Architecture Pattern and Rationale

A layered architecture with a controller-based design is used. The Presentation layer contains the Console UI and input handling. The Application layer contains the Quiz Controller, Quiz Logic, Score/Result, and Admin Module. The Data layer contains Question Data. This separation keeps responsibilities clear and makes individual modules easier to test and maintain.

A single large main function was not selected because it would mix input, quiz logic, scoring, and data handling. A client-server or microservices architecture was also not selected because the project is a local console application with no network or multi-user requirement.

3.6 Technology Stack & Data Stores

Item	Choice	Purpose
Programming Language	C++17	Implementation of the console quiz application.
Standard Library	C++ Standard Library	Input/output, strings, vectors, file handling and basic data structures.
Build/Compiler	GCC / MinGW / compatible C++17 compiler	Compilation and execution on supported platforms.
Question Data	Local question file / data structure	Stores quiz questions, options and correct answers.
Result Data	Local file/output	Stores or displays quiz results as required by the implementation.

3.7 Risks & Mitigations

Risk	Mitigation
Invalid user input	Validate menu and answer choices and ask the user to enter a valid value again.
Incorrect score calculation	Keep answer checking and score calculation in the Quiz Logic/Score component and test it independently.
Missing or invalid question data	Validate question data before use and display an error instead of terminating unexpectedly.
Unauthorized question modification	Separate administrator functions from normal player functions and require administrator verification.
Corrupted or tampered question data	Validate question records before loading and apply the security controls defined in the SRS.

3.8 Traceability to Requirements

The architecture is traced to the Online Quiz System SRS using the following requirement groups. Functional requirements OQ-F-001 to OQ-F-016 are implemented through the Console UI, Quiz Controller, Quiz Logic, Score/Result, Question Data and Admin Module. Non-functional requirements OQ-NF-001 to OQ-NF-005 are addressed through input validation, modular design, response-time goals, usability and portability. Security requirements OQ-SR-001 to OQ-SR-005 are addressed through bounded input, administrator authorization, protection of question data, and controlled result handling.

Requirement Group	Main Design Components	Design Sections
OQ-F-001 to OQ-F-005	Console UI, Quiz Controller, Input Handling	3.4, 4.1, 4.2
OQ-F-006 to OQ-F-011	Quiz Logic, Score/Result, Quiz Controller	3.4, 4.1, 4.2
OQ-F-012 to OQ-F-016	Admin Module, Question Data, Console UI	3.4, 4.2, 4.3

OQ-NF-001 to OQ-NF-005	All relevant components	3.5, 3.6, 3.7, 4.4, 4.5
OQ-SR-001 to OQ-SR-005	Input Handling, Admin Module, Question Data, Result handling	3.9, 4.4

3.9 Security Architecture

The security design focuses on input validation, administrator authorization, protection of question data, and safe result handling. Player input is treated as untrusted and is validated before being processed. Administrator functions are separated from normal quiz functions so that question-management operations are not available through the normal player flow. Question data is validated before use, and sensitive answer information is not exposed during the quiz.

Security objectives are confidentiality of answer information, integrity of question and result data, and prevention of malformed input from causing unsafe program behaviour. These objectives support OQ-SR-001 to OQ-SR-005.

4. Design

4.1 Design Overview

The system is divided into small modules with clear responsibilities. The Console UI communicates with the Quiz Controller. The Quiz Controller coordinates the quiz flow and calls Quiz Logic for question retrieval, answer checking and scoring. Question Data provides the question information, while Score/Result prepares the final result. The Admin Module handles question management separately from the player flow.

4.2 UML Sequence Diagrams

The following sequence diagrams show two important flows required by the SAD template: taking a quiz and administrator question management.

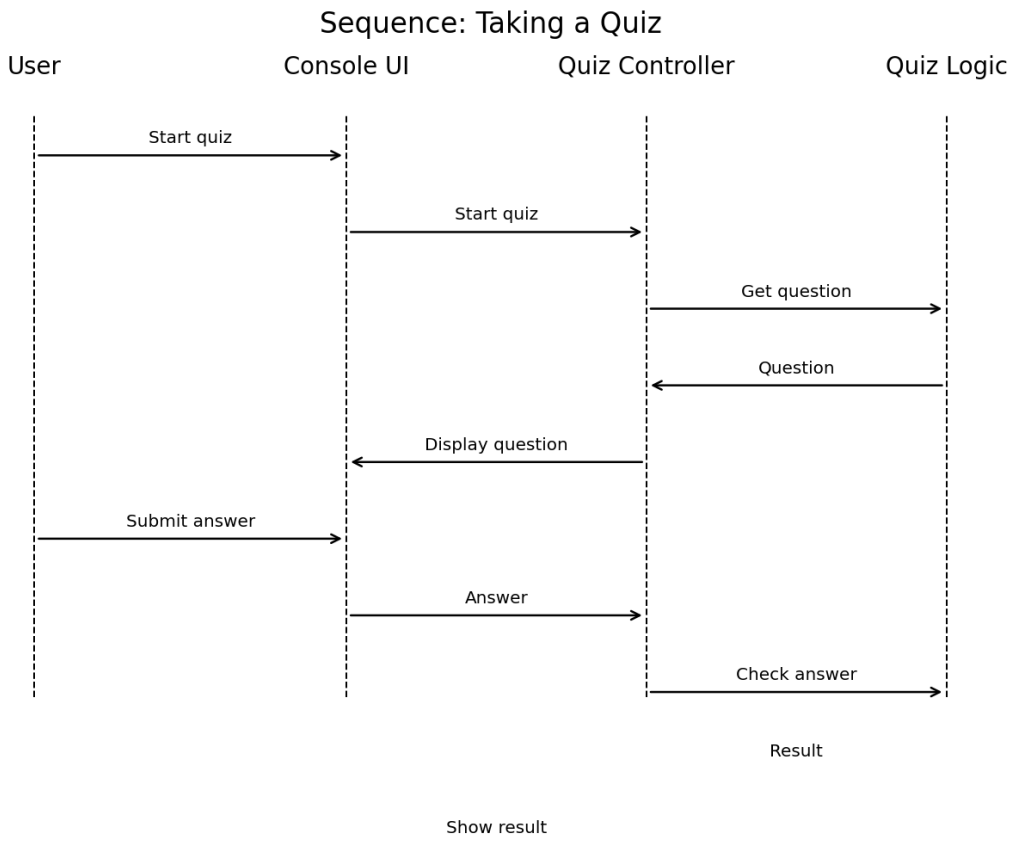


Figure 4.1 – UML Sequence Diagram: Taking a Quiz

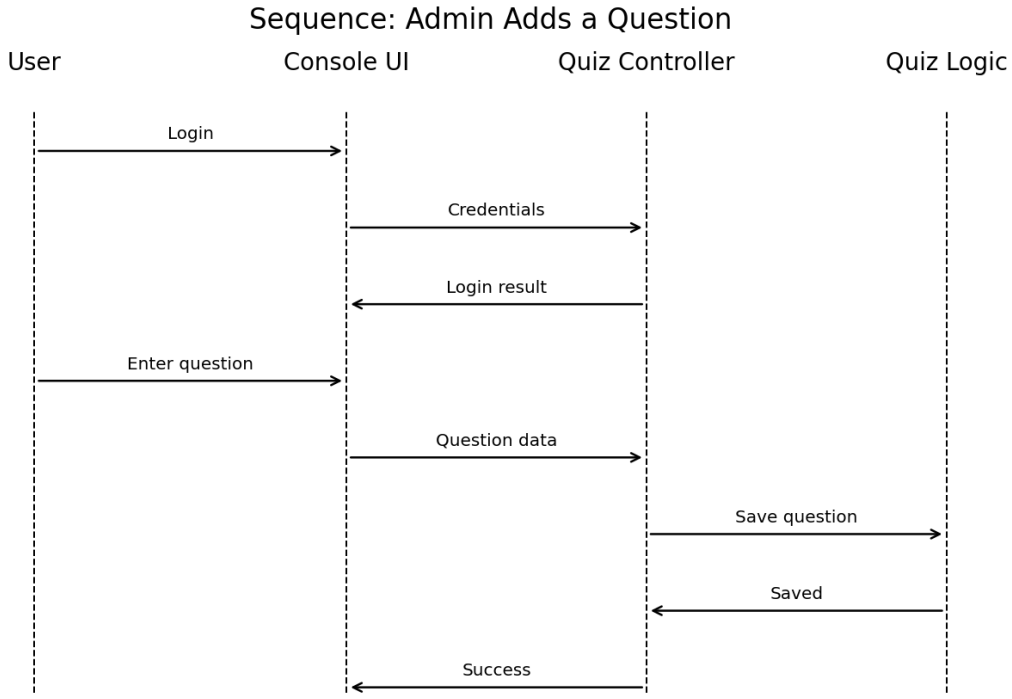


Figure 4.2 – UML Sequence Diagram: Administrator Adds a Question

4.3 API Design

Because the Online Quiz System is a console application, its APIs are internal C++ interfaces rather than web endpoints.

Component / Function	Input	Output / Behaviour
QuizController::startQuiz()	Quiz/question data	Starts the quiz session and coordinates question flow.
QuizLogic::getQuestion()	Question index / quiz state	Returns the question and available options.
QuizLogic::checkAnswer()	Question and selected option	Returns whether the selected option is correct.
ScoreResult::calculateScore()	Answer results	Calculates correct count, percentage and final result.
AdminModule::addQuestion()	Question text, options, correct answer	Validates and stores a new question.

4.4 Error Handling, Logging & Monitoring

Invalid menu choices and answer options are rejected with a clear message and the user is prompted again. Missing, empty or invalid question data is detected before it is used. The application should return to a safe menu state rather than terminate unexpectedly when recoverable errors occur. Administrator operations should report success or failure clearly. If result logging is implemented, only necessary result information should be stored and user input should be validated before writing it.

4.5 UX Design

The console interface presents one question at a time with clearly labelled options. Instructions are shown before the quiz starts. The user receives immediate feedback after submitting an answer, and the final score is displayed after the quiz. Administrator functions use a separate menu/flow so that question management is distinct from the player experience.

4.6 Open Issues & Next Steps

• A graphical user interface can be added in a future version.

• A database can replace local question storage if the project scope is expanded.

• Timed quizzes and user accounts can be added as future enhancements if required.

• Additional automated tests can be added for larger question banks and edge cases.

5. Appendices

5.1 Glossary

Term	Meaning
MCQ	Multiple Choice Question
UI	User Interface
SRS	Software Requirements Specification
SAD	Software Architecture and Design Specification
RTM	Requirements Traceability Matrix

5.2 References

1. Software Requirements Specification (SRS) – Online Quiz System.

2. Software Engineering Mini-Project Deliverables Part-1.

3. SAD Template provided for the Software Engineering mini-project.

5.3 Tools

Microsoft Word / compatible document editor; C++17 compiler; UML diagramming tools as used for the project diagrams.